1. Problem Statement

Design a centralised rate-limiting component for an e-commerce platform's API layer. The rate limiter sits in front of all API services and enforces per-user request limits within a sliding time window — preventing abuse, protecting downstream services from traffic spikes, and ensuring fair usage across all clients.

Rate limiting is not a product visible to end users; it is infrastructure middleware. Its correctness, latency, and reliability directly affect the availability and integrity of every API it protects.

1.1 Functional Requirements

- Enforce a maximum of 100 requests per minute per user (configurable per tier).
- Identify users by IP address for unauthenticated (guest) requests and by user ID or API key for authenticated requests.
- Reject requests that exceed the limit immediately — do not queue or delay them.
- Return HTTP 429 Too Many Requests with a Retry-After header indicating seconds remaining in the current window.
- Rate limits reset automatically at the end of each time window — no manual unblock process required.
- Support differentiated limits per user tier (free, premium, enterprise).

1.2 Non-Functional Requirements

- High throughput — the rate limiter must handle 10,000 requests per second across all users at peak.
- Low latency — the allow/reject decision must add minimal overhead to every request, since it sits in the critical path of every API call.
- High availability — the rate limiter must not become a single point of failure for the platform.
- Accuracy — limits must be enforced consistently across all distributed instances, not per-instance.

1.3 Out of Scope

- Permanent account bans or manual block lists — rate limiting here is window-based throttling only.
- DDoS mitigation at the network layer — this design addresses application-layer rate limiting only.

- Per-endpoint rate limits — a single per-user limit is applied across all endpoints for the current version.

2. Capacity Estimation

The capacity numbers drive the most critical design choices — particularly the algorithm selection and the Redis HA strategy.

Metric	Value	Implication
Peak requests/sec (all users)	10,000 req/sec	Rate limiter must process 10,000 Redis operations per second — latency per operation must be sub-millisecond
Rate limit per user	100 requests / 60 seconds	Each user has a sliding window counter; TTL-based reset eliminates the need for a separate reset job
Redis operations per request	2–3 (fetch config, fetch counts, INCR)	Redis must sustain ~30,000 ops/sec at peak — well within Redis's single-node capacity (~100k ops/sec)
Counter keys per active user	3 keys (config, prev_count, curr_count)	Memory per user: ~200 bytes; 1M active users ≈ ~200MB — trivial for Redis

Key implication

At 10,000 req/sec, every millisecond of latency added by the rate limiter compounds across all traffic. This rules out any algorithm requiring per-request database queries or large in-memory scans. Redis's sub-millisecond atomic operations make it the most appropriate choice for the target throughput and latency requirements.

3. Algorithm Selection

Four rate-limiting algorithms were evaluated against the specific constraints of this system: 10,000 req/sec throughput, Redis-backed shared state, and an e-commerce context where burst traffic from bots or broken clients is a primary concern.

Algorithm	How it works	Memory cost	Verdict
Fixed Window	Counter resets at fixed time boundaries (e.g. every 60s)	1 counter per user	Rejected — edge burst vulnerability: a client can send 2× the limit by

			straddling a window boundary
Sliding Window Log	Stores the timestamp of every request; counts only those within the last 60 seconds	1 entry per request per user	Rejected — memory cost scales with traffic volume, not just user count; requires a timestamp list scan per request
Token Bucket	Tokens refill at a steady rate; each request consumes one token	2 values per user (tokens + last refill time)	Viable but not optimal — intentionally allows burst traffic, which is undesirable for an e-commerce order/payment API
Sliding Window Counter	Blends prev + current window counts using a weighted overlap ratio	2 counters per user	Selected — eliminates edge burst problem, minimal memory, fast O(1) calculation, no burst tolerance needed here

3.1 Sliding Window Counter — Mechanics

The algorithm maintains two counters per user in Redis and calculates an estimated request count for the current sliding window on every request:

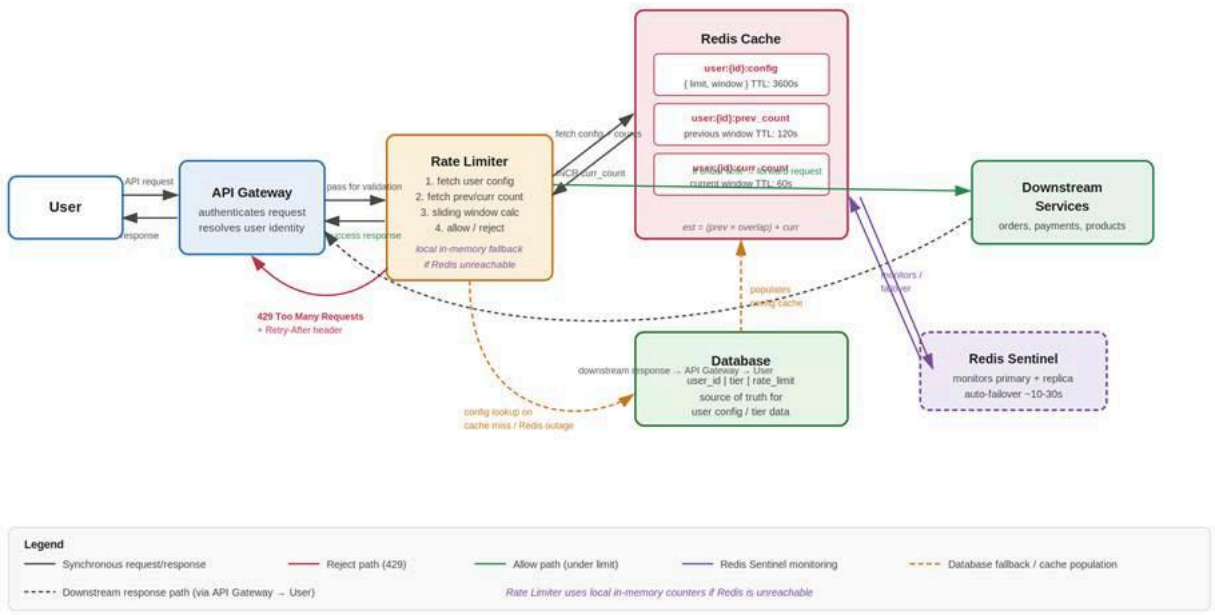
```
elapsed = current_time - window_start
overlap = (window_seconds - elapsed) / window_seconds
estimated   = (prev_count × overlap) + curr_count

if estimated < limit  →  allow, INCR curr_count
if estimated ≥ limit  →  reject with HTTP 429
```

Example: a user's window started 20 seconds ago (40 seconds remaining from the previous window). If prev_count = 100 and curr_count = 40: estimated = (100 × 40/60) + 40 = 66.7 + 40 = 106.7 → over limit → reject. This prevents the double-burst that Fixed Window allows, without storing individual request timestamps.

4. High-Level Architecture

The rate limiter is implemented as middleware within the API Gateway — the single entry and exit point for all platform traffic. This placement is necessary because user identity (required for per-user rate limiting) is resolved during the API Gateway's authentication step; a rate limiter placed before the API Gateway would only have access to IP addresses, losing per-user granularity for authenticated requests.



4.1 Components

Component	Responsibility
API Gateway	Single entry and exit point for all traffic. Authenticates requests and resolves user identity (IP for guests, user ID for authenticated users) before passing to Rate Limiter. All responses — including 429 rejections — return to the user through the API Gateway.
Rate Limiter	Middleware inside the API Gateway. Fetches user config and sliding window counters from Redis, performs the weighted calculation, and makes the allow/reject decision. Maintains a local in-memory counter as fallback when Redis is unreachable.
Redis Cache	Shared state store for all Rate Limiter instances. Holds three key types per user: config (limit/window), prev_count, and curr_count. All atomic operations (INCR, EXPIRE, TTL) execute in sub-millisecond time.

Redis Sentinel	Background HA monitor. Watches the Redis primary and replica; automatically promotes the replica to primary on failure (~10-30 seconds). Not in the request path — runs entirely in the background.
Downstream Services	The API services being protected (orders, payments, products, etc.). Only receive requests that the Rate Limiter has explicitly allowed.
Database	Persistent source of truth for user account data — stores user_id, tier, and rate_limit. Redis caches this config; the database owns it. The Rate Limiter queries the database directly on a Redis cache miss or during a Redis outage.

Note: In this design, the API Gateway's built-in routing capability handles load distribution across multiple Rate Limiter instances running in active-active configuration. A separate load balancer component is not required — the API Gateway natively manages upstream instance selection, eliminating an additional infrastructure dependency.

4.2 Database — Source of Truth for User Config

The database is not in the primary request path. Under normal operation, the Rate Limiter reads user config exclusively from Redis. The database is accessed in exactly two scenarios:

- Cache miss on user:{id}:config — the first request from a user whose config has not yet been cached, or after the 1-hour TTL has expired. The Rate Limiter queries the database, retrieves the user's tier and limit, caches the result in Redis under user:{id}:config with a 3600s TTL, then proceeds with the rate limiting calculation.
- Redis outage — if Redis is completely unreachable, the Rate Limiter falls back to local in-memory counters for the sliding window state and queries the database directly for the user's limit threshold on each request.

The database does not store request counters or sliding window state — that is exclusively Redis's responsibility. The database's role is narrow and deliberate: own the user config, serve it when Redis cannot.

5. Redis Data Model

Three key types are maintained per user in Redis. All are sourced originally from the database and use TTL-based expiry — no separate reset job or scheduled process is required.

Key	Value	TTL	Purpose
user:{id}:config	{ limit: 100, window: 60 }	3600s (1 hour)	User's tier limit — sourced from database on first request, cached for 1 hour. Supports tiered limits without a DB hit on every request.

user:{id}:prev_count	Integer	120s	Previous window's final request count. Used in the weighted overlap calculation. TTL of 2× window ensures it outlives the current window for calculation purposes.
user:{id}:curr_count	Integer	60s	Current window's running request count. INCR'd on every allowed request. When TTL fires, this key is promoted to prev_count and reset to 0 for the next window.

Why TTL instead of a reset job

A separate program checking every second and sending counter resets would introduce a race condition: requests arriving in the gap between a window boundary and the reset program firing could be incorrectly counted against the expired window. Redis TTL eliminates this entirely — the key expires atomically at the exact moment the window ends, with no coordination required between the rate limiter instances and an external process.

6. Request Flow

6.1 Happy Path (Under Limit)

1. User sends API request to the API Gateway.
2. API Gateway authenticates the request and resolves user identity (user ID or IP address).
3. Rate Limiter fetches user:{id}:config from Redis to determine the applicable limit. On cache miss, fetches from the database and caches the result.
4. Rate Limiter fetches prev_count, curr_count, and window_start from Redis.
5. Rate Limiter calculates estimated = (prev_count × overlap) + curr_count.
6. Estimated count is under the limit — Rate Limiter executes INCR user:{id}:curr_count (atomic) and forwards the request to the Downstream Service.
7. Downstream Service processes the request and returns a response.
8. Response travels back through the API Gateway to the User.

6.2 Unhappy Path (Limit Exceeded)

9. Steps 1–5 same as above.

10. Estimated count meets or exceeds the limit.
11. Rate Limiter rejects the request immediately — no forwarding to Downstream Service.
12. Rate Limiter calculates Retry-After value using Redis TTL command on user:{id}:curr_count (seconds remaining until window reset).
13. Rate Limiter returns the rejection to the API Gateway.
14. API Gateway sends HTTP 429 response to the User:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 47
Content-Type: application/json

{ "error": "Too many requests. Try again in 47 seconds." }
```

7. High Availability and Failure Strategy

7.1 The Distributed Counter Problem

At 10,000 req/sec, multiple Rate Limiter instances run simultaneously behind the API Gateway (active-active redundancy). A single user's requests are distributed across all instances by the load balancer. Without a shared counter store, each instance would maintain its own local count — Instance A might see 40 requests from User X, Instance B sees 35, Instance C sees 30. The real total is 105 (over the limit), but no single instance knows that.

Redis solves this with atomic INCR operations — all Rate Limiter instances write to the same shared counter, and Redis processes each increment one at a time regardless of concurrency. The count is always accurate across all instances.

7.2 Redis High Availability — Redis Sentinel

Redis is a critical dependency — both the sliding window counters and the user config cache live there. A Redis outage would affect every rate limiting decision. Redis Sentinel provides automatic HA:

- A Redis primary handles all writes; a replica maintains a live copy of all data.
- Redis Sentinel continuously monitors both nodes.
- On primary failure, Sentinel automatically promotes the replica to primary within approximately 10-30 seconds.
- Rate Limiter instances are configured with the Sentinel endpoint — they query Sentinel to discover the current primary address, so they automatically reconnect to the promoted replica after failover.

Redis Cluster was considered and rejected for this use case — it adds significant operational complexity (cross-slot coordination, cluster topology management) without providing a benefit

that Redis Sentinel doesn't cover at our scale. Sentinel provides sufficient HA with far lower complexity.

7.3 Failure Strategy — Fail-Open with In-Memory Fallback

Two failure strategies were evaluated for Redis unavailability:

Strategy	Behavior during Redis outage	Risk
Fail closed	Reject all requests until Redis recovers	Entire platform unavailable to all users — including legitimate customers — even though downstream services are healthy
Fail open	Allow all requests through without rate limiting	Abusive traffic could overwhelm downstream services during the outage window

For an e-commerce platform where uptime directly affects revenue and customer trust, fail closed is not acceptable. However, pure fail-open leaves the platform unprotected during an outage. The chosen approach is a pragmatic middle ground:

- If Redis is unreachable, each Rate Limiter instance activates its local in-memory counter for that user.
- Local counters apply a conservative limit (50% of normal) — users are lightly throttled but not blocked.
- Limits are approximate during this window (instances can't coordinate locally), but protection is maintained against extreme abuse.
- On Redis recovery, Rate Limiter instances resume Redis-backed counters; local counters are discarded.
- Redis Sentinel's automatic failover (~10-30 seconds) minimizes the window during which the fallback is active.

8. Tiered Rate Limits

Since requests are authenticated and user identity is resolved at the API Gateway, the Rate Limiter has access to the user's account context on every request. This enables differentiated limits per tier without any additional infrastructure:

Tier	Rate limit	Config TTL in Redis
Free	100 requests / minute	3600s
Premium	1,000 requests / minute	3600s

Enterprise	10,000 requests / minute	3600s

The user's limit is stored in the database as the source of truth and cached in Redis under user:{id}:config with a 1-hour TTL. On every request, the Rate Limiter reads the limit from Redis (or falls back to the database on cache miss) before performing the sliding window calculation. If a user upgrades their tier, the new limit propagates within 1 hour when the cached config expires and is refreshed from the database.

9. Summary and Design Principles

This rate limiter design makes three non-obvious choices worth highlighting as the key learnings from this exercise:

- **Algorithm choice is driven by use case, not convention.** Token Bucket is widely used and well-known, but its burst tolerance is a liability for transactional APIs. Sliding Window Counter was selected because its trade-offs (slightly approximate, two counters per user) precisely match the constraints (memory efficiency, high throughput, no burst needed).
- **TTL is more reliable than a reset job.** A separate scheduled reset process introduces a race condition at window boundaries. Redis TTL eliminates the race entirely — the key expires atomically at the window boundary with no external coordination.
- **Fail strategy depends on what you're protecting.** The initial instinct was fail-open for e-commerce uptime. The introduction of Redis Sentinel changed the framing — the real answer is 'make Redis reliable enough that the failure mode rarely matters,' with a conservative in-memory fallback as a safety net for the rare outage window.